11-768 AI Agents
Assignment 3: Training Agents with Reinforcement Learning

Experimental report · Fall 2026

Name: Yubo Chen

Andrew ID: yuboc

This report contains the requested analyses and diagnostic plots for all seven formal training runs, followed by the four full-split TextCraft evaluations. Parts 0 and 1 provide the runtime understanding and implemented data pipeline; their source code is included separately in the submission.

Report contents

Section	Study
Part 2	Synchronous REINFORCE and GRPO
Part 3	Asynchronous GRPO: timing, drift, and real rollouts
Part 4.1	Group size, useful data, and advantage variance
Part 4.2	Synchronous versus asynchronous training
Part 4.3	Importance weighting under higher staleness
Part 5	TextCraft-Synth training and medium/hard evaluation

Reading the results

Each configuration was run once, using its supplied settings unchanged. Figures show recorded values without smoothing; missing update diagnostics remain gaps. Learner iterations and actual optimizer updates are distinguished, and each pass@k curve names its exact k. Part 4.2 explicitly reuses Parts 2–3. Part 5 retains abandoned episodes as failures and distinguishes periodic 50-task evaluation from the final complete validation splits.

Public final adapter

https://huggingface.co/Marstbowl/scottyrl-a3-p5_textcraft_grpo

Base: Qwen/Qwen3.5-4B · LoRA rank: 32 · Final sampler: v50

Part 2: Synchronous REINFORCE and GRPO

Experimental setup

I ran the provided p2a_number_search_reinforce and p2b_number_search_grpo configurations unchanged on Number Search. Both runs started independently from Qwen/Qwen3.5-4B with LoRA rank 32, learning rate 10^{-5}, gradient clipping at 1.0, temperature 1.0, and seed 11768. Thinking was disabled and thinking history was preserved. Each run used eight tasks per training iteration and completed 50 iterations, with baseline evaluation at iteration 0 and evaluation every 10 iterations on the first 50 validation tasks, four rollouts per task. Each method was run once.

Setting	p2a: REINFORCE	p2b: GRPO
Rollouts per task	1	8
Rollouts per training iteration	8	64
Episode advantage	Raw reward	Reward minus its group mean
Loss	Unclipped importance sampling	PPO
Training pass metric	train/pass@1	train/pass@8
Evaluation pass metric	eval/pass@4	eval/pass@4

The successful p2b run used a fresh backend after a previous attempt failed while saving its baseline snapshot, before any evaluation or optimizer update. That zero-update attempt was archived and is excluded from the results below; the configurations were not changed during recovery. The metrics and figures use only the two completed formal runs, not the short probe.

Results

Metric	p2a: REINFORCE	p2b: GRPO
Initial evaluation success rate	35.5%	34.5%
Evaluation success rate at iteration 20	47.0%	98.0%
Final evaluation success rate	52.0%	99.0%
Initial eval/pass@4	76.0%	74.0%
Final eval/pass@4	86.0%	100.0%
Recorded training iterations	50	50
Optimizer updates	50	34
Skipped iterations	0	16
Time after baseline, including periodic evaluations	50.31 min	32.89 min

The timing interval starts at the recorded iteration-0 baseline result and ends at the final iteration-50 evaluation result. It excludes initial model startup, baseline evaluation, and the failed p2b startup attempt. All recorded training and evaluation abandoned-episode counts were zero, and the final sampler paths and resumable checkpoints were saved for both runs.

Learning-speed comparison

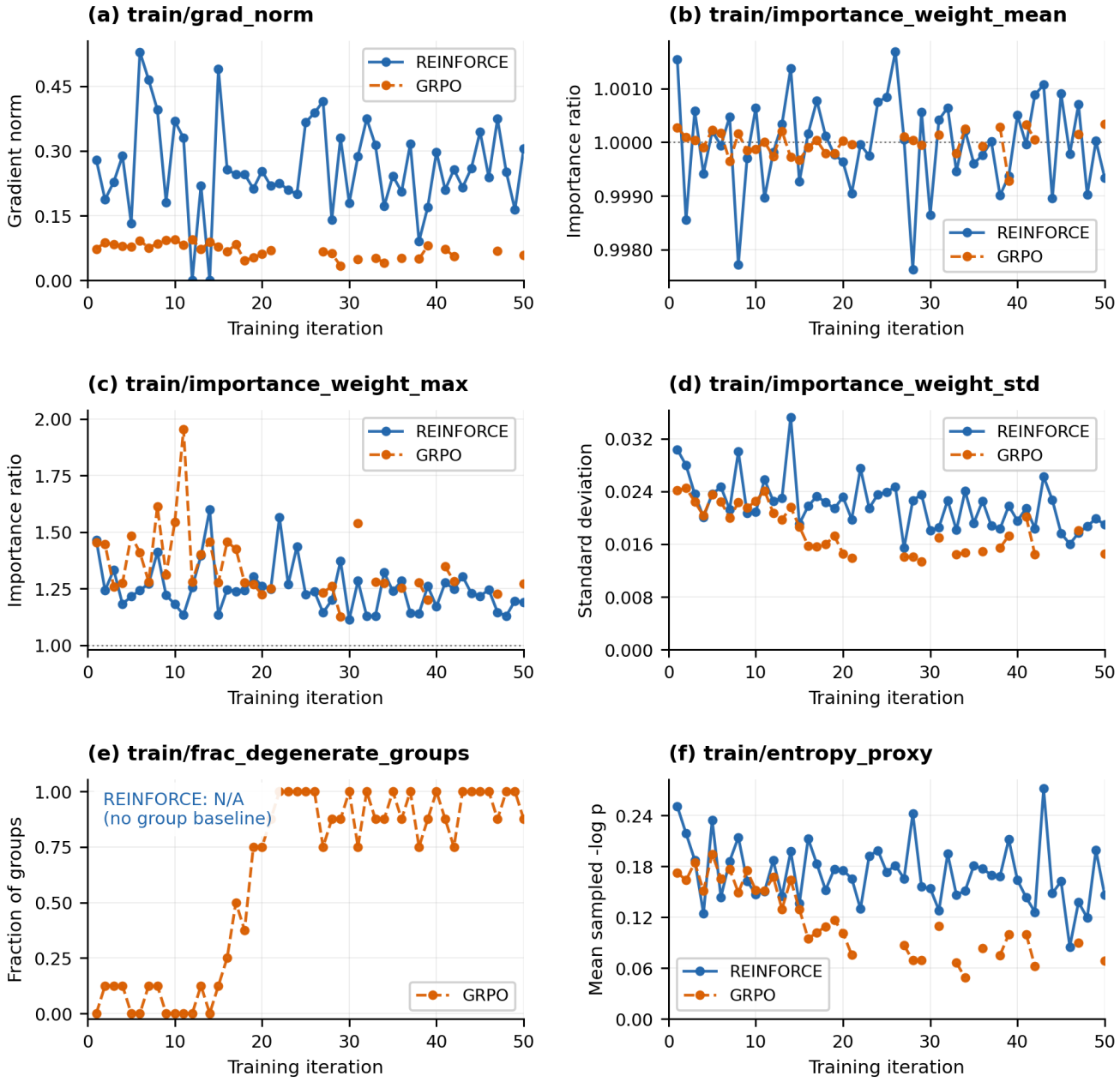
Under the provided configurations, GRPO reached 98.0% validation success by iteration 20 and 99.0% by iteration 50, while REINFORCE reached 47.0% and 52.0%, respectively, from comparable untrained baselines of 34.5% and 35.5%. The faster rise for GRPO is consistent with the expected benefit of comparing several attempts at the same task: centered advantages can reinforce above-average attempts and suppress below-average ones, whereas REINFORCE uses the nonnegative binary reward directly. The configurations also differ in rollout count (eight versus one per task) and loss (PPO versus unclipped importance sampling), so this comparison cannot isolate the group baseline or establish superior sample efficiency. GRPO subsequently saturated on this validation subset, and its all-success groups produced 16 skipped iterations, which explains part of its shorter recorded run without showing that an individual GRPO update is inherently faster.



Raw single-run metrics; step 0 is the untrained evaluation baseline.

Figure 1. Reward quality versus training iteration. The plots show the unsmoothed train/success_rate, eval/success_rate, train/pass@1 for REINFORCE, train/pass@8 for GRPO, and eval/pass@4 for both runs. The small eight-episode REINFORCE training batches fluctuate substantially; its train/pass@1 equals its training success rate by definition. GRPO's train/pass@8 measures whether any of eight attempts succeeds, so it must not be interpreted as the same quantity as REINFORCE's train/pass@1; the common evaluation metrics provide the direct held-out comparison. Both evaluation success curves finish above their baselines, and GRPO's eval/pass@4 reaches 100% from iteration 20 onward, while REINFORCE finishes at 86%. Training rewards describe the pre-update sampling policy, whereas evaluation at the same iteration describes the current policy after that iteration's update.

Part 2 · Diagnostic plots (2/2)



Gaps in GRPO update diagnostics are skipped iterations; no values are filled with zero.

Figure 2. Optimization, policy drift, and data health versus training iteration. The panels show train/grad_norm, train/importance_weight_mean, train/importance_weight_max, train/importance_weight_std, train/frac_degenerate_groups, and train/entropy_proxy. Mean importance weights remain close to 1 in both synchronous runs; their maxima stay below 2 and their standard deviations below 0.036, indicating small measured drift on action tokens. Gradient norms remain finite and below 0.53; their different magnitudes should be interpreted with the different advantages, batch sizes, and losses in mind. The action-token surprise proxy falls overall from 0.251 to 0.146 for REINFORCE and from 0.173 to 0.069 for GRPO, consistent with more predictable outputs, but this proxy alone does not establish policy collapse. For GRPO, it describes only retained, nondegenerate groups. GRPO's degenerate-group fraction rises as its groups become uniformly successful, reaching 1 on the 16 fully skipped iterations; the REINFORCE fraction is N/A because it has no group baseline, even though its logger records zero. Missing gradient, importance-weight, and entropy values at skipped GRPO iterations are left missing rather than replaced with zeros; those iterations remain present in the reward curves and training log.

Part 3: Asynchronous GRPO

Setup and learning outcome

I ran p3_number_search_async unchanged, independently from the base Qwen/Qwen3.5-4B model, with LoRA rank 32, PPO, learning rate 10^{-5}, and seed 11768. Eight rollout workers feed a queue of capacity 16; each learner iteration accepts eight groups of eight rollouts, with maximum staleness 2. The server splits two H100s between training and sampling. Evaluation uses the first 50 validation tasks, four rollouts each, at baseline and every ten iterations. The comparisons below use the completed formal p2b and p3 runs; probes are excluded. Each configuration was run once.

Metric	p2b: synchronous	p3: asynchronous
Initial eval/success_rate	34.5%	31.5%
Final eval/success_rate	99.0%	99.5%
Final eval/pass@4	100%	100%
Iterations / optimizer updates	50 / 34	50 / 40
Skipped iterations	16	10
Mean train/step_time_s	38.03 s	32.13 s
Median train/step_time_s	27.92 s	27.95 s
Sum of iteration times	31.69 min	26.78 min
Baseline result to final evaluation	32.89 min	28.07 min
Time-weighted learner-wait fraction	66.3%	35.7%

p3's validation success rises from 31.5% to 44.0%, 80.0%, 98.5%, 99.0%, and 99.5% at iterations 10 through 50. It initially learns more slowly per iteration than p2b, which reaches 98% at iteration 20, but both finish near 100%. The final difference is one successful episode among 200, so this single-run comparison does not establish a quality advantage.

Wall-clock comparison

Using all 50 logged train/step_time_s values, including skipped iterations, the speedup is 38.0287 / 32.1329 = 1.1835, or 15.5% less time per iteration. The median is essentially unchanged: the speedup describes average and total time, not every individual iteration. The timing intervals start after the baseline result and run between training records. They include first-iteration and checkpoint overhead, and synchronous periodic evaluations enter the following iteration's interval. They exclude cold start, baseline evaluation, and the final evaluation drain; the latter adds 71.76 s for p2b and 77.46 s for p3.

The time-weighted learner-wait fraction, sum(learner_wait_s) / sum(step_time_s), falls from 66.3% to 35.7%, consistent with overlapping rollouts and background evaluation with optimization. This residual measures time outside updates and sampler saves, not GPU utilization. It is not the arithmetic mean of per-iteration fractions, which would overweight near-instant skipped iterations. The comparison also includes different GPU roles (two colocated training/sampling GPUs versus one GPU for each role), task order and realized samples, 34 versus 40 actual updates, and 1,200 versus 1,720 retained sequences. It therefore measures the provided complete systems rather than an isolated scheduling optimization.

Stale data and completion

p3 accepted 400 groups and discarded 71 stale groups: 15.1% of the 471 dequeued groups, not of all generated groups. Queue leftovers and in-flight groups at shutdown are outside these counters. Importance weighting corrects the accepted older-policy data, with the ratio tails discussed in Figures 2 and 3. There is no observed final-quality penalty relative to p2b, although learning per iteration differs. All abandoned-episode counts are zero. Ten all-success batches carry no GRPO signal and skip updating, so 50 iterations produce version 40; the final sampler is v40 and the last resumable state is iteration 49 / update 40, before the skipped final iteration.

Early, middle, and late training behavior

The following three real training rollouts come from runs/p3_number_search_async_trajectories/. Their logged step identifies the completion interval, while policy_version identifies the snapshot that generated them. H, L, and C below mean too high, too low, and correct. These are illustrative examples on different tasks, not a controlled estimate of average search efficiency.

Early: step 5, policy version 4

Task number_search.train.65 has range [301, 697], target 471, and a nine-guess budget. Source: step_000005 / train_number_search.train.65_rollout0.jsonl.

Guesses: 500 H → 350 L → 450 L → 480 H → 440 L → 460 L → 470 L → 475 H → 470 L. The episode exhausts all nine guesses and receives reward 0. After learning that the target exceeds 450, the agent tries 440; later it repeats 470 despite already receiving “too low” for that value. These are strategically inconsistent guesses, not parser or argument errors.

Middle: step 25, policy version 24

Task number_search.train.272 has range [142, 669], target 242, and a ten-guess budget. Source: step_000025 / train_number_search.train.272_rollout0.jsonl.

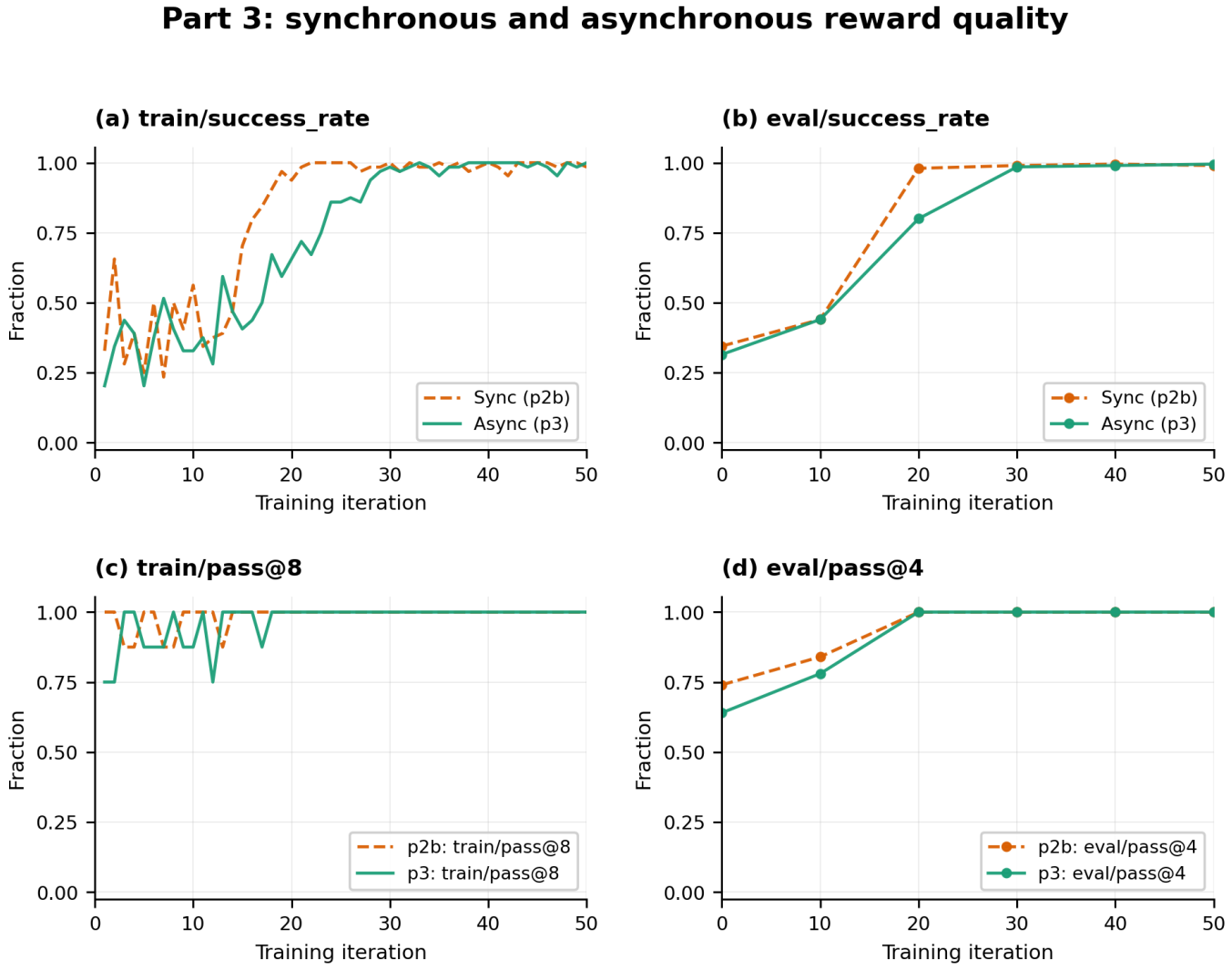
Guesses: 405 H → 283 H → 221 L → 252 H → 236 L → 244 H → 240 L → 242 C. It succeeds in eight guesses, receiving reward 1. Every guess lies within the interval consistent with earlier feedback, although the second and third probes differ from that interval's midpoint.

Late: step 45, policy version 36

Task number_search.train.428 has range [538, 892], target 862, and a nine-guess budget. Source: step_000045 / train_number_search.train.428_rollout0.jsonl.

Guesses: 715 L → 804 L → 848 L → 871 H → 859 L → 865 H → 862 C. It succeeds in seven guesses, receiving reward 1. Every action chooses a floor or ceiling midpoint of the integer interval still consistent with all previous feedback.

Two concrete changes appear in these examples: better respect for feedback-derived bounds, and more systematic bisection. The number of guesses outside the feasible interval falls from two in the early example to zero in the middle and late examples. Midpoint choices account for 2/9, 6/8, and 7/7 actions, respectively. Bounds are updated using all earlier feedback, so an inconsistent guess never widens the feasible interval. These examples support the behavioral interpretation alongside the held-out reward improvement, while the differing tasks prevent attributing their exact guess counts solely to training.



Unsmoothed formal runs; iteration 0 is the evaluation baseline.

Figure 1. Reward quality versus training iteration. Both formal runs show train/success_rate, eval/success_rate, train/pass@8, and eval/pass@4, without smoothing. Training success improves markedly in both runs and later approaches 1. The higher training pass@8 curves measure whether at least one of eight attempts succeeds, rather than the average success of an episode; evaluation pass@4 uses a different k. p3 reaches evaluation pass@4 = 1 at iteration 20, while its per-episode evaluation success continues rising to 99.5%. The slower initial rise in p3's evaluation success relative to p2b is visible despite their similar final results. Train rewards describe accepted pre-update rollouts, potentially from older versions; asynchronous evaluations use the snapshot captured at the labeled iteration and may finish later. Iteration-50 evaluation uses p3 version 40 because skipped iterations do not advance the policy.

Part 3 · Diagnostic plots (2/3)

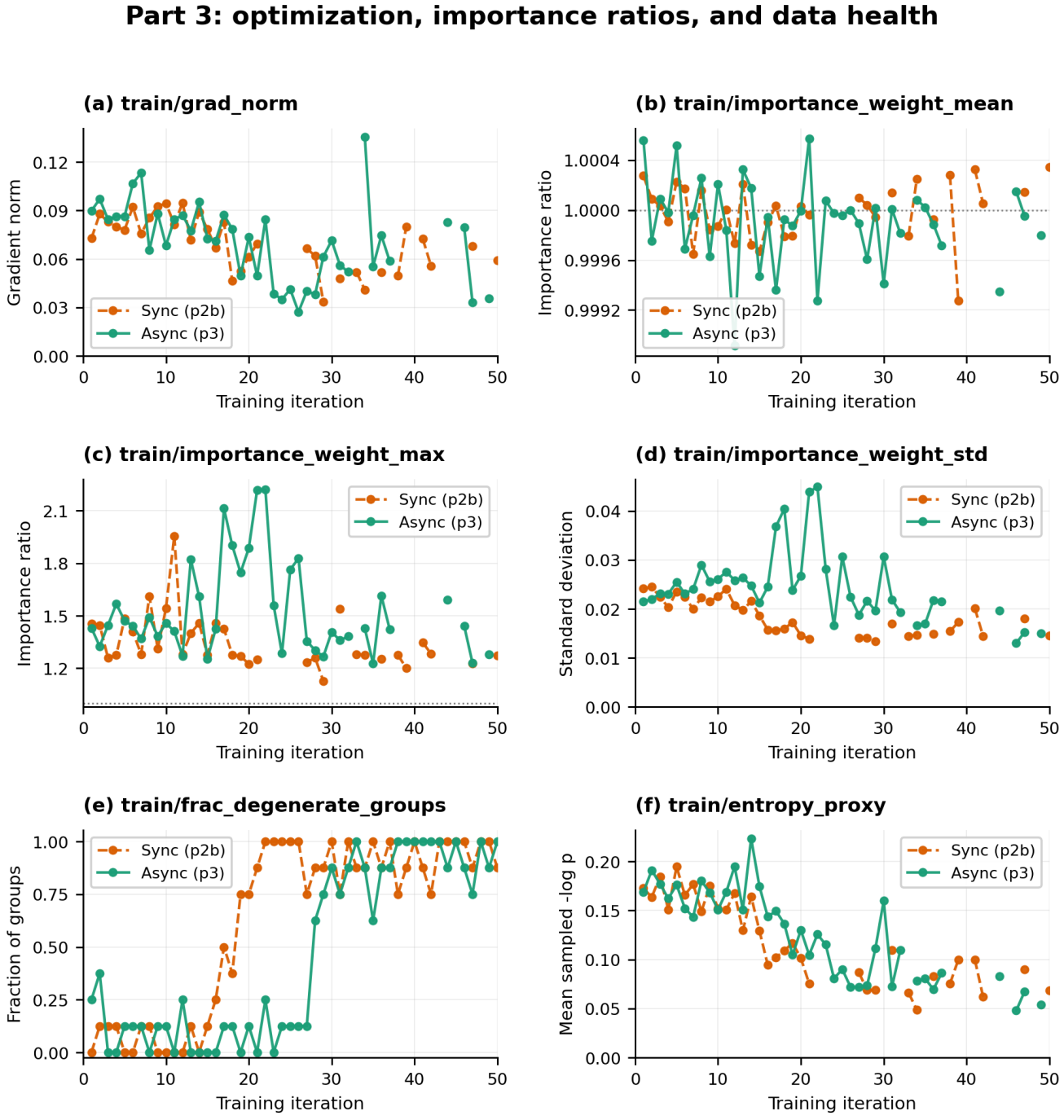
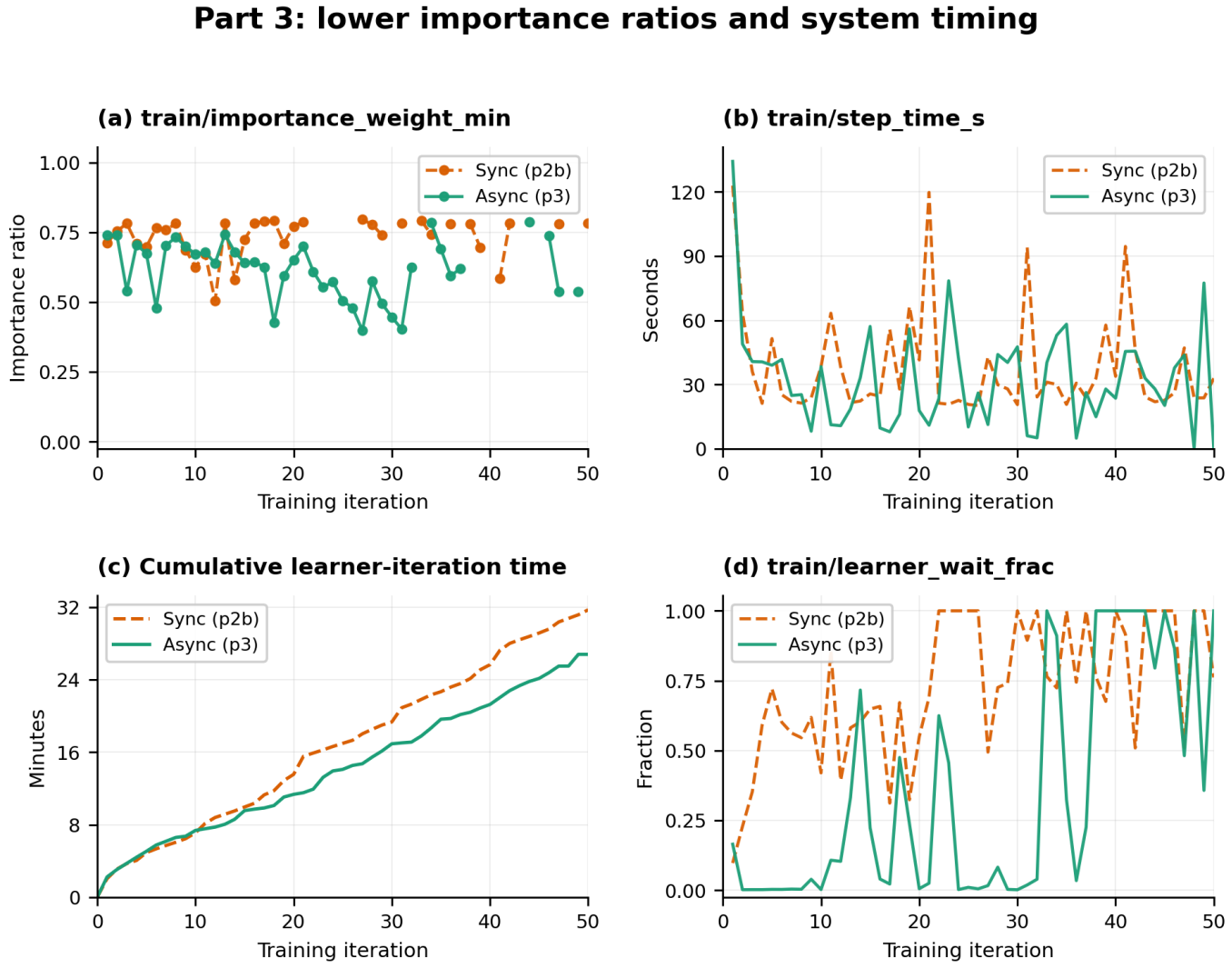


Figure 2. Optimization, importance ratios, and data health. Gradient norms remain finite: 0.0335–0.0946 for p2b and 0.0272–0.1355 for p3. The per-update IW means remain near 1 (p2b: 0.999274–1.000343; p3: 0.998915–1.000574), but p3 has a wider tail: its maximum reaches 2.219 versus 1.954, and its highest standard deviation is 0.04494 versus 0.02450. The minima in Figure 3 complete this comparison; the mean alone would hide these differences. These action-token statistics are raw ratios, so values outside PPO's clipping interval are expected and do not mean clipping failed. Degenerate fractions rise with uniformly successful groups; the 16 p2b and 10 p3 fully degenerate iterations skip updates and retain gaps in gradient, IW, and entropy curves. The action-token surprise proxy decreases from 0.173 to 0.069 in p2b and from 0.169 to 0.054 in p3. It describes retained nondegenerate groups and predictable format tokens as well as decisions, so its decline alone does not establish collapse.

Part 3 · Diagnostic plots (3/3)



Timing includes all 50 iterations; cumulative time excludes final evaluation drain.

Figure 3. Lower importance ratios and timing. The minimum IW falls as low as 0.504 in p2b and 0.398 in p3, complementing p3's wider upper tail and standard deviation in Figure 2. This is measurable sampling/training mismatch without a runaway tail in these runs. Raw step times retain long iterations and very short skipped iterations; their similar medians do not contradict the lower asynchronous mean. Cumulative time sums all 50 iteration intervals and ends at 31.69 versus 26.78 minutes, before final evaluation drain. The plotted learner-wait fraction is the raw per-iteration value; it can equal 1 on a skipped iteration that takes almost no wall time. The report table therefore uses time-weighted fractions, 66.3% versus 35.7%, to summarize reduced waiting. Overlap and background evaluation are consistent with this reduction, while the differences in GPU roles, actual updates, and retained batches limit causal attribution.

Part 4.1: Group Size

Setup and results

I compare the completed formal p2a, p41, and p2b runs, using their provided configs unchanged. Each starts independently from Qwen/Qwen3.5-4B, with LoRA rank 32, learning rate 10^{-5}, seed 11768, eight task groups per iteration, and 50 iterations. p2a uses REINFORCE, raw rewards as advantages, and the unclipped importance-sampling loss; p41 and p2b use group-mean-centered GRPO advantages and PPO. Evaluation uses the same first 50 validation tasks with four rollouts each. Each configuration was run once; probes are excluded.

Metric	p2a: G=1	p41: G=2	p2b: G=8
Initial eval/success_rate	35.5%	35.5%	34.5%
Iteration-20 eval/success_rate	47.0%	44.5%	98.0%
Final eval/success_rate	52.0%	95.5%	99.0%
Final eval/pass@4	86.0%	100%	100%
Optimizer updates / skipped iterations	50 / 0	41 / 9	34 / 16
Sampled training episodes	400	800	3,200
Retained training sequences	400	282	1,200
Retained action targets	47,088	40,479	244,375
Full-run degenerate fraction	N/A	64.75%	62.50%
First-10-iteration degenerate fraction	N/A	55.00%	6.25%
First-10 mean retained sequences / iteration	8.0	7.2	60.0

G=2 improves strongly, but much later than G=8: its evaluation success is 35.0%, 44.5%, 60.5%, 94.0%, and 95.5% at iterations 10 through 50. G=8 already reaches 98% at iteration 20. REINFORCE ends at 52%, below either GRPO run. This is a descriptive comparison, not a pure group-size ablation: p2a changes both the advantage rule and loss, and the configs sample 8, 16, or 64 episodes per iteration. More progress per iteration therefore does not establish greater sample efficiency.

The usefulness of a GRPO group depends on having different rewards, not merely on completing episodes. During the first ten iterations, mean training success is similar for G=2 and G=8 (42.5% versus 41.1%), yet 44/80 G=2 groups are degenerate versus 5/80 G=8 groups. The remaining 36 pairs provide 72 sequences, whereas 75 eight-rollout groups provide 600. Thus G=2 loses a much larger fraction of its already smaller batch before learning. Retained sequence counts here match retained episodes; in general the data pipeline can split an episode into multiple sequences.

Whole-run degeneracy alone hides this early difference. G=8 quickly becomes successful, so 245 of its 250 degenerate groups are all-success and only five are all-failure. G=2 has 173 all-success and 86 all-failure degenerate groups. Late all-success groups explain why G=8 finishes with a similar total degenerate fraction despite learning much faster. Its smaller number of optimizer updates still processes many more retained sequences. All nine skipped G=2 iterations are all-success batches; iterations 47–50 skip, so the final evaluation legitimately uses the unchanged iteration-46 sampler.

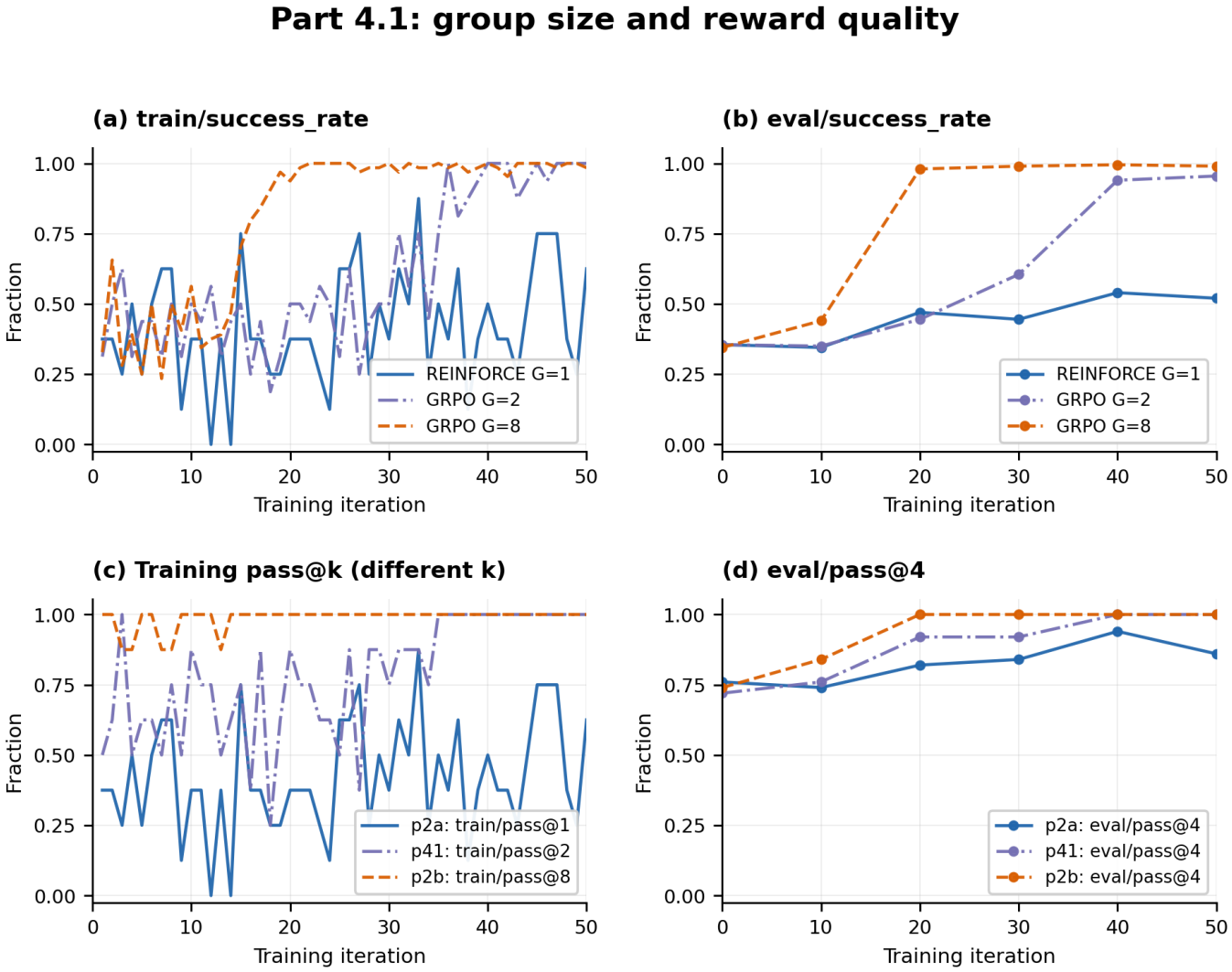
What changes in advantage variance?

For illustration, condition on a fixed task and policy with independent binary rewards of success probability p. An equal-reward group occurs with probability p^G + (1-p)^G; smaller groups are more likely to be degenerate. The group-mean baseline has variance p(1-p)/G, so G=2 provides a noisier estimate of expected reward than G=8. These formulas explain the direction of the data-loss effect without treating the heterogeneous, changing training tasks as one fixed-p experiment.

The variance of an individual centered advantage requires more care because its own reward is included in the baseline. Before filtering, Var(r_i - group_mean) = p(1-p)(1 - 1/G), which is actually smaller for smaller G at fixed p. Thus it would be incorrect to say that every definition of advantage variance necessarily increases when G shrinks. Baseline-estimation noise, unconditional centered-advantage variance, and the variance of the retained training signal are different quantities.

After filtering, a retained G=2 binary group is always one success and one failure, with advantages +0.5 and -0.5 and equal-episode-weight variance 0.25. For a mixed G=8 group with k successes, that variance is (k/8)(1-k/8), at most 0.25. These retained comparisons omit all zero-signal degenerate groups. The logged train/adv_std adds another distinction: sequence_batch_stats pools retained action tokens, so long episodes carry more weight than short ones. Its mean over recorded updates is 0.489 for G=2 versus 0.381 for G=8. This is consistent with the selected G=2 signal having large magnitude, but is not an estimate of unconditional episode-level variance or proof of lower gradient-estimator variance for G=8.

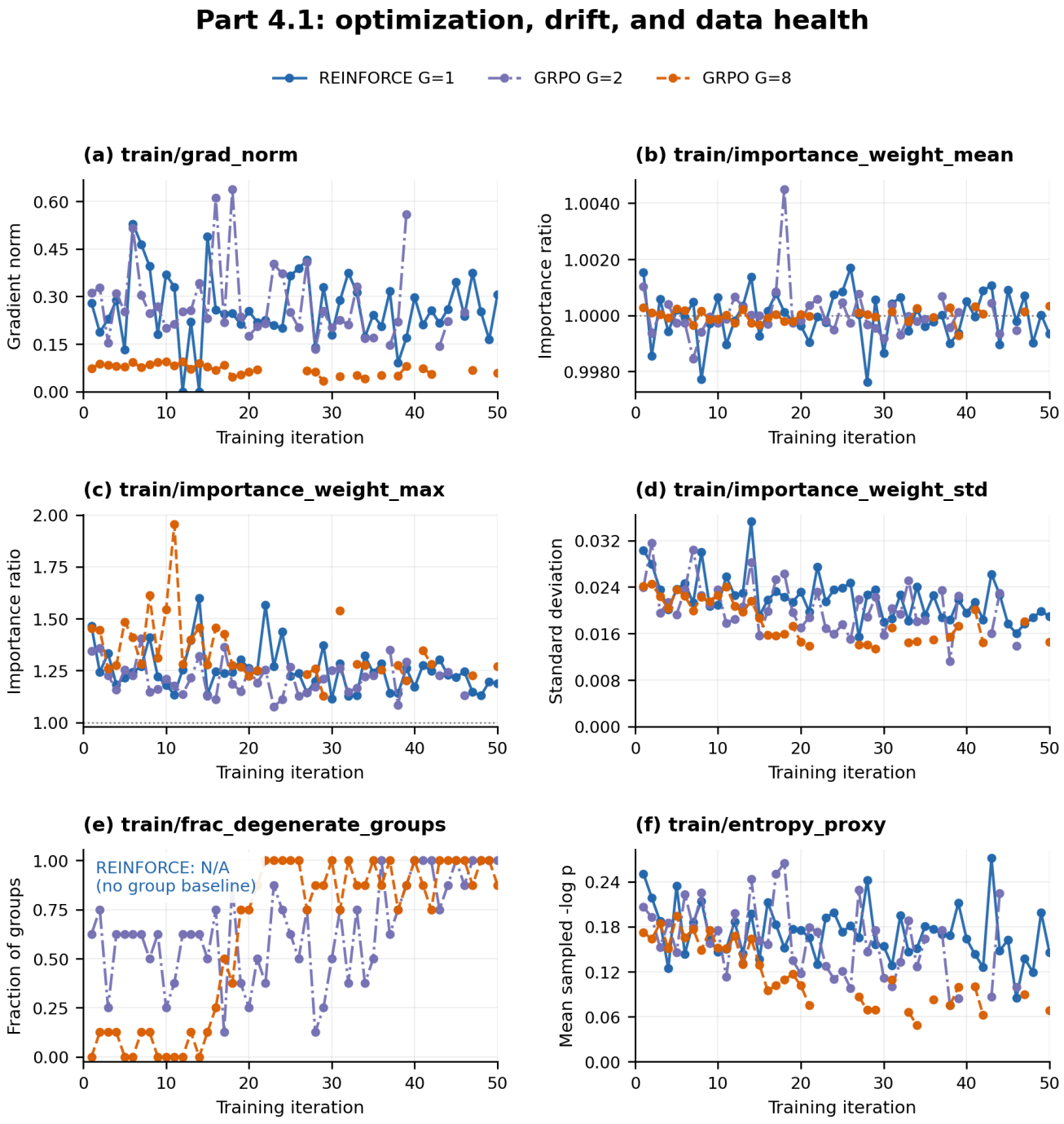
REINFORCE has no group baseline, so its degenerate fraction is N/A even though its logger emits 0. Its 400 retained episodes include 231 failures with zero advantage; only the 169 successful episodes have a nonzero reward-weighted signal. GRPO instead drops equal-reward groups, leaving 282 G=2 and 1,200 G=8 sequences with nonzero binary centered advantages. For fixed p, G=2 has a noisier baseline estimate; in these runs it also has larger average absolute advantages across retained action tokens and fewer useful sequences per iteration than G=8. The curves support slower early learning with G=2, while the changing policy, unequal rollout budgets, and single runs prevent an isolated causal or sample-efficiency conclusion.



Raw formal runs; training pass@1, @2 and @8 are distinct quantities.

Figure 1. Reward quality versus iteration. All unsmoothed train/success_rate and eval/success_rate curves are shown with every logged pass key: p2a train/pass@1, p41 train/pass@2, p2b train/pass@8, and each run's eval/pass@4. Training success rises earlier for G=8 and later for G=2; REINFORCE fluctuates without a comparable rise. Training pass@1 equals episode success, whereas pass@2 and pass@8 ask whether any attempt in a group succeeds and tend to be higher. Their different k values preclude treating them as the same metric. Evaluation pass@4 reaches 1 at iteration 20 for G=8 and iteration 40 for G=2, while per-episode success continues improving; REINFORCE ends at 0.86. Training curves use pre-update rollouts and evaluations use the snapshot at the labeled iteration. G=2 finishes at 95.5% episode success, G=8 at 99.0%, and REINFORCE at 52.0%.

Part 4.1 · Diagnostic plots (2/3)



Skipped update diagnostics stay missing; REINFORCE degeneracy is N/A.

Figure 2. Optimization, importance ratios, and data health. G=2 gradient norms span 0.134–0.637, compared with 0–0.529 for REINFORCE and 0.0335–0.0946 for G=8; their finite scales alone do not rank learning quality. IW means remain near 1 (G=2: 0.99846–1.00450), while maximum ratios reach 1.600, 1.403, and 1.954 for G=1/2/8. The highest IW standard deviations are 0.03526, 0.03156, and 0.02450, respectively, so neither the mean nor group size alone determines the tail. These are raw ratios; PPO clipping does not bound the logged ratios. Degeneracy is N/A for REINFORCE; the G=2/G=8 curves show the early data-loss difference and late all-success saturation described above. The action-token surprise proxy declines from 0.251 to 0.146, 0.207 to 0.100, and 0.173 to 0.069 for G=1/2/8. It includes predictable formatting and, for GRPO, only retained groups; declining values alone do not establish collapse. Missing update diagnostics remain gaps, while genuine zeros remain plotted.

Part 4.1 · Diagnostic plots (3/3)

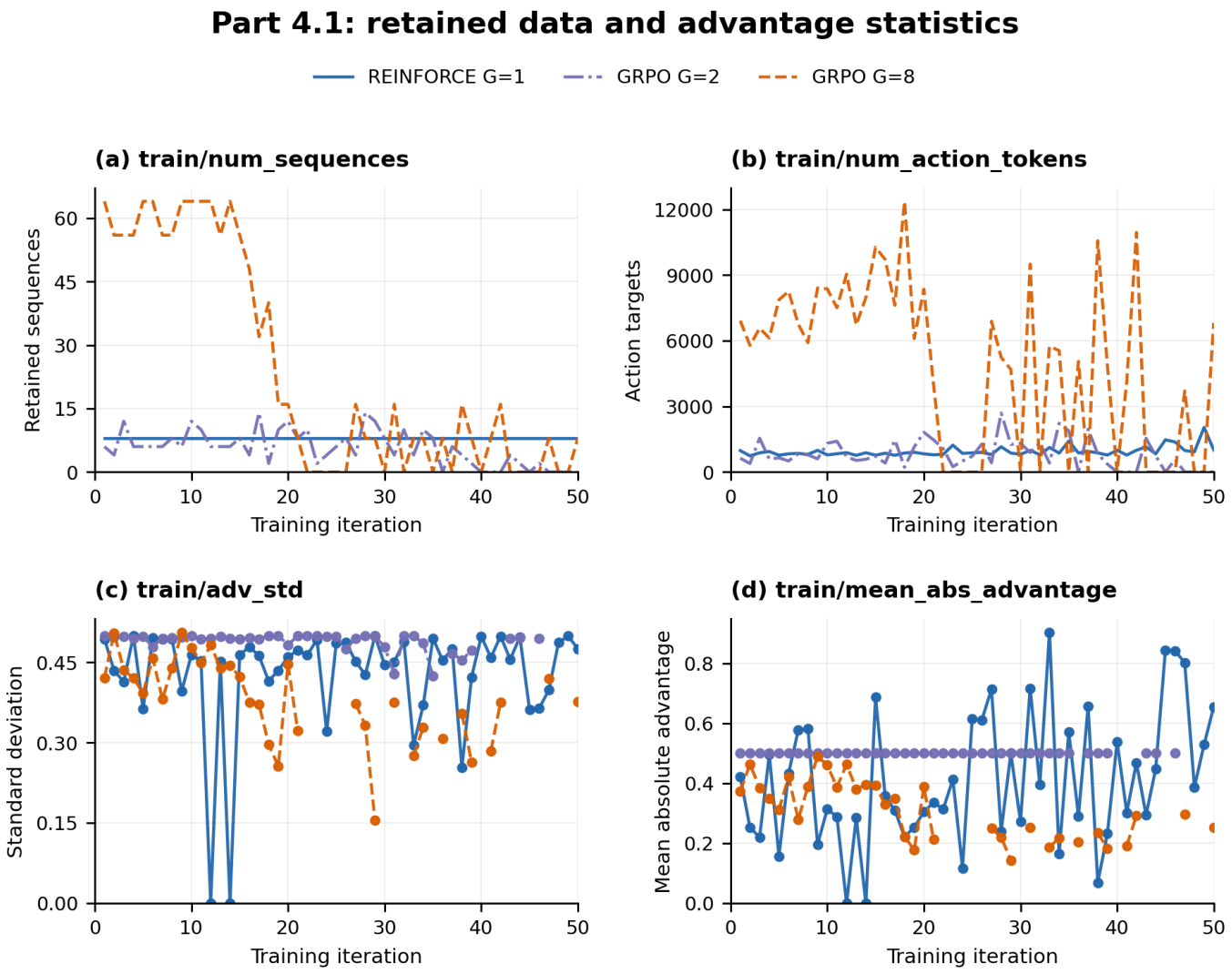


Figure 3. Retained data and advantage statistics. Retained sequence counts show G=8's much larger early learning batches, G=2's smaller mixed-pair batches, and REINFORCE's constant eight sequences, including zero-advantage failures. Action-target counts also depend on episode length, so their totals need not have the same ratios as sequence counts. Fully degenerate GRPO iterations have real zeros in both data-volume panels. The train/adv_std and train/mean_abs_advantage panels instead contain gaps at empty batches. G=2 always has absolute advantage 0.5; token-length imbalance can reduce its logged standard deviation below 0.5. G=8 uses group-dependent centered rewards, with mean absolute advantage averaging 0.310 and logged standard deviation averaging 0.381 across its 34 updates. REINFORCE uses raw 0/1 rewards, making mean absolute advantage the action-token-weighted success fraction rather than a centered statistic. These conditional token-weighted moments must be distinguished from the fixed-task unconditional variance discussed above.

Part 4.2: Synchronous vs. Asynchronous Training

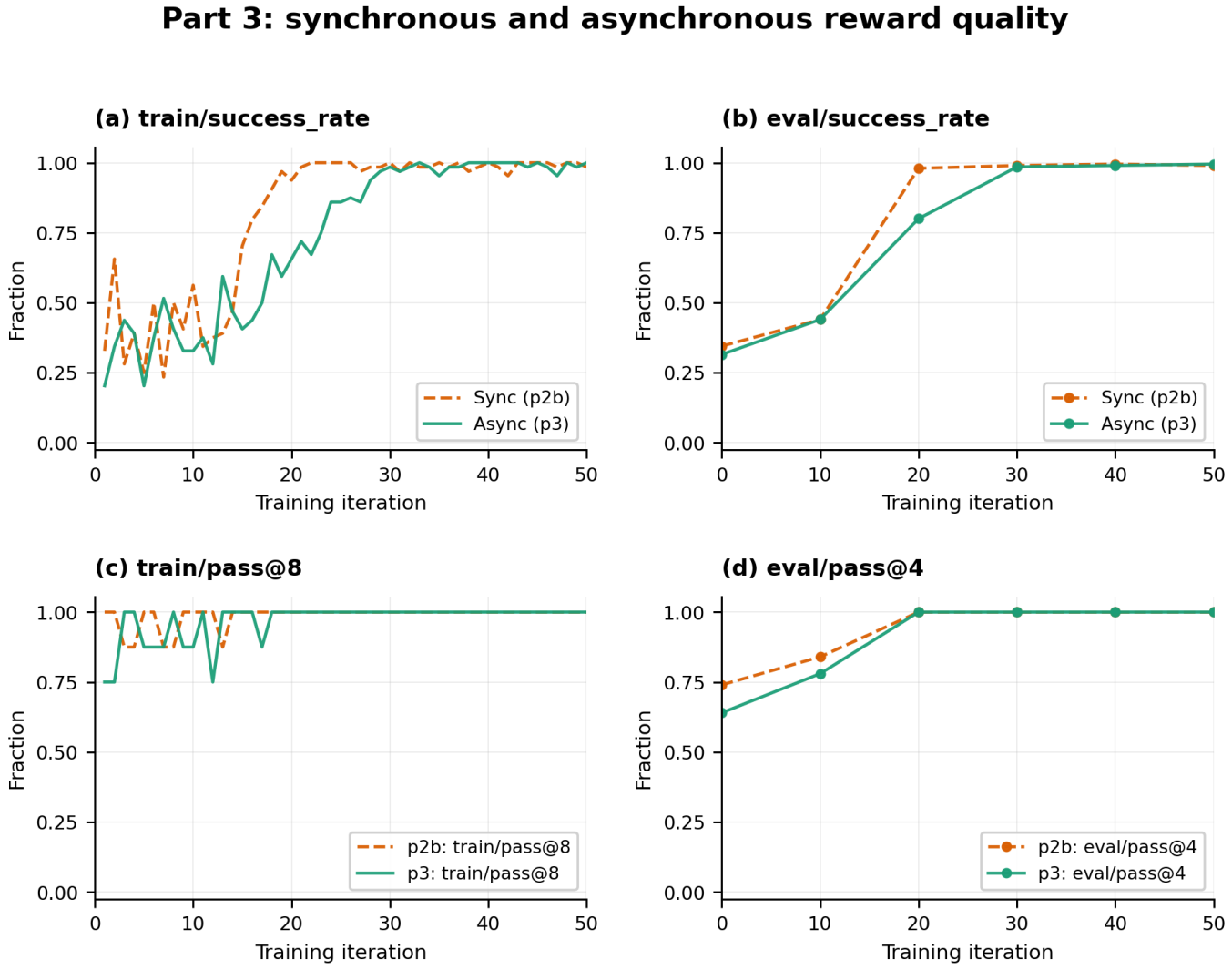
Results and interpretation

This comparison reuses the completed formal p2b and p3 runs from Parts 2 and 3, with no new training or probe results. Both use GRPO with eight rollouts per task, PPO, and 50 learner iterations; evaluation uses 50 validation tasks with four rollouts each. p3 uses eight workers, a queue of capacity 16, and maximum staleness 2. Each configuration was run once. The standard diagnostic figures below are reused from Part 3.

Metric	p2b: synchronous	p3: asynchronous
Final eval/success_rate	99.0%	99.5%
Final eval/pass@4	100%	100%
Optimizer updates / skipped iterations	34 / 16	40 / 10
Retained training sequences	1,200	1,720
Mean train/step_time_s	38.03 s	32.13 s
Median train/step_time_s	27.92 s	27.95 s
Sum of iteration intervals	31.69 min	26.78 min
Time-weighted learner-wait fraction	66.3%	35.7%

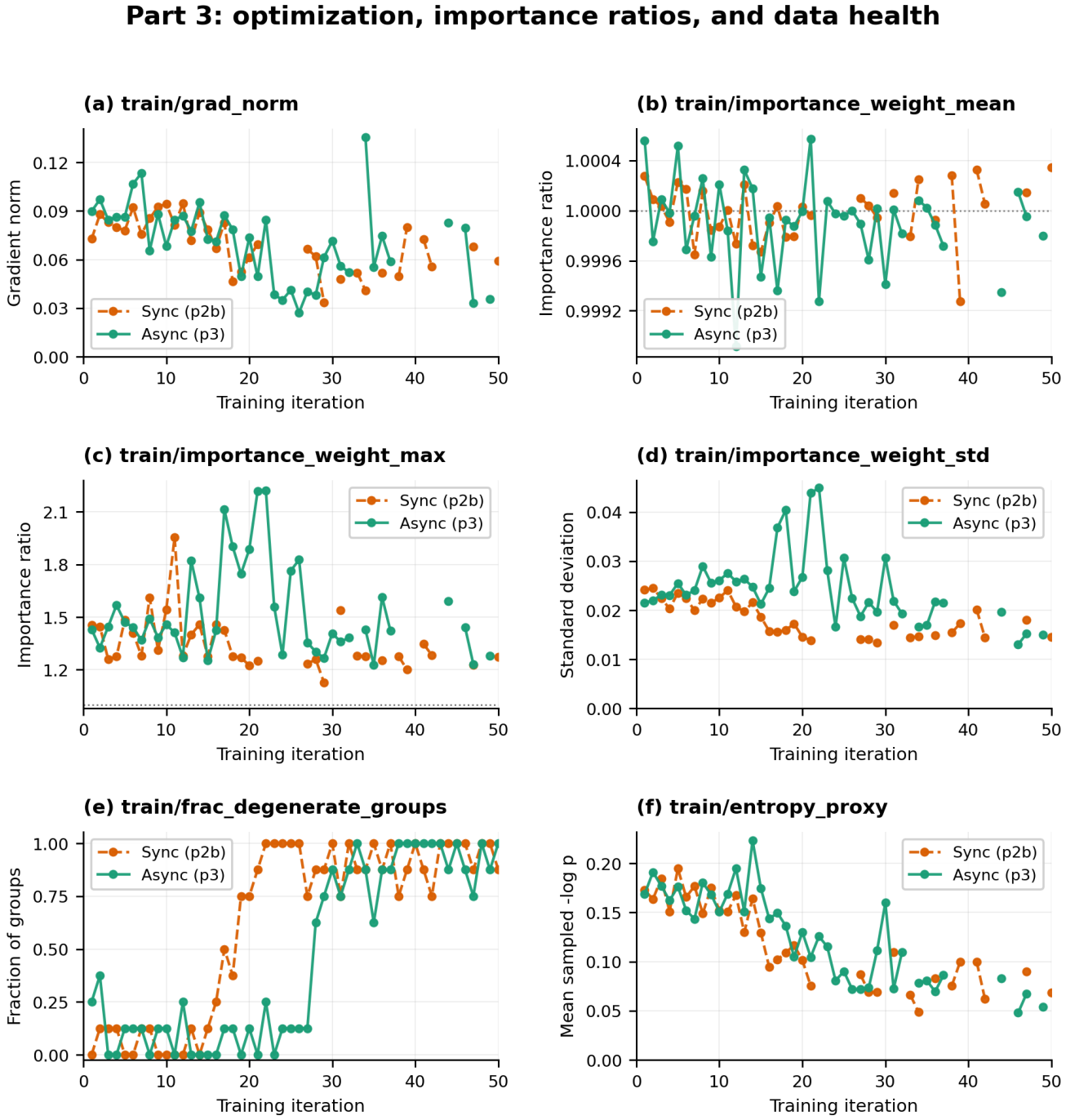
Across all 50 iterations, including skipped ones, asynchronous training achieves 38.0287 / 32.1329 = 1.1835 times the speed, or 15.5% less time per iteration. The intervals include first-iteration and checkpoint overhead; synchronous periodic evaluation enters the next interval, while asynchronous evaluation overlaps training. Cold start, baseline evaluation, and the final evaluation drain are excluded. Median step times are almost identical, so the improvement concerns the average and total rather than every iteration. The time-weighted learner-wait fraction, sum(learner_wait_s) / sum(step_time_s), falls from 66.3% to 35.7%, consistent with overlapping rollouts and evaluation with optimization. This residual is time outside updates and sampler saves, not GPU utilization. Different GPU roles, task order, actual updates, and retained sequence counts limit attribution to scheduling alone: p2b colocates training and sampling on two GPUs, whereas p3 assigns one GPU to each role.

p3's final summary reports 71 stale groups discarded and 400 accepted: 15.1% of 471 dequeued groups, excluding queue leftovers and in-flight work at shutdown. Accepted older-policy data use PPO importance ratios. The per-update IW means remain near 1 in both runs, but p3 has wider tails: minimum 0.398 versus 0.504, maximum 2.219 versus 1.954, and highest standard deviation 0.04494 versus 0.02450. Thus the mean alone would hide the measured drift. p3 initially learns more slowly per iteration, reaching 80% evaluation success at iteration 20 versus p2b's 98%, but finishes at 99.5% versus 99.0%. There is no observed final-quality penalty in these runs; the final difference is only one episode among 200 and does not establish superiority. These observations show the tradeoff between overlap, stale-data handling, and learning progress without proving which individual mechanism caused the final result.



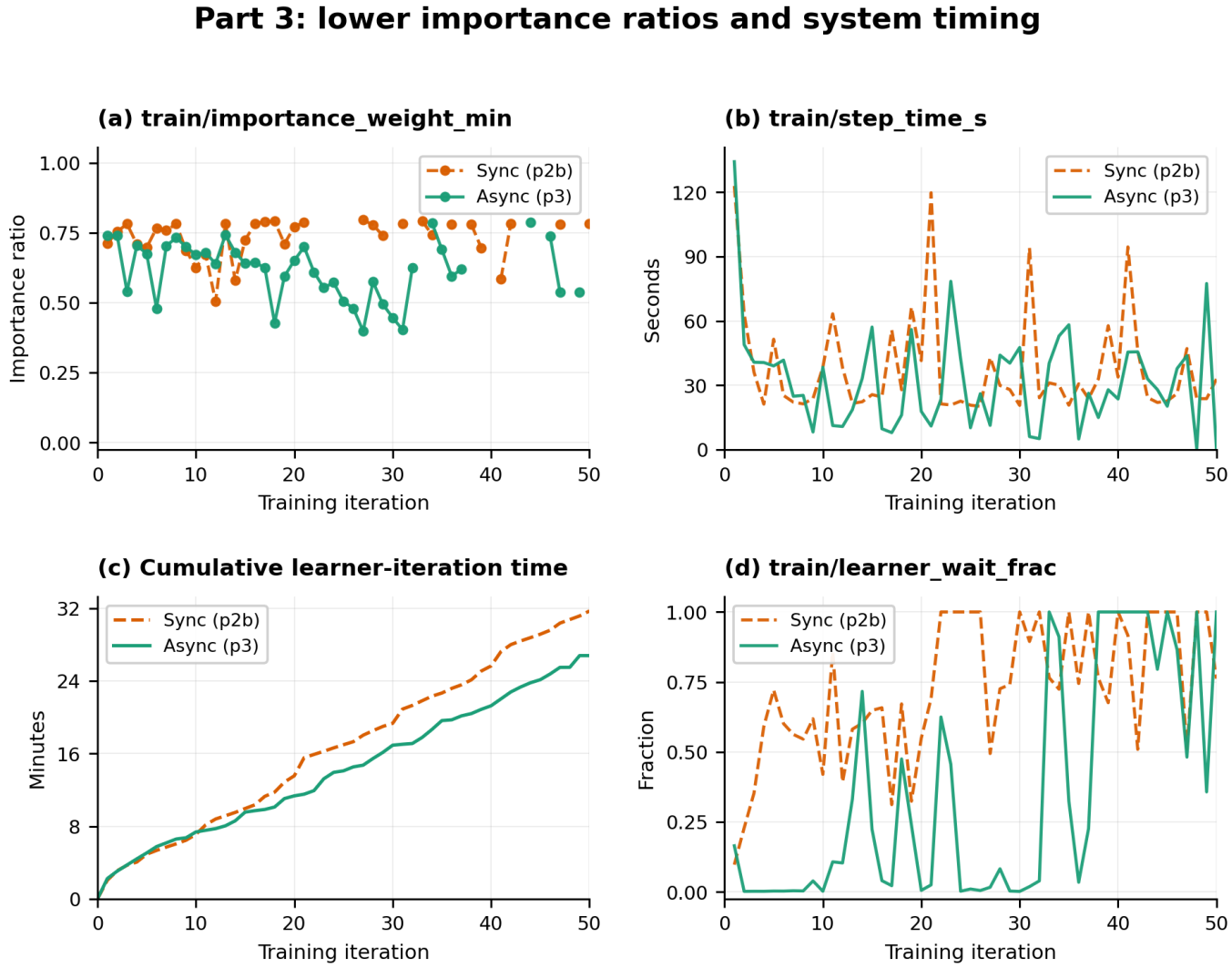
Unsmoothed formal runs; iteration 0 is the evaluation baseline.

Figure 1. Reward quality; reused from Part 3, Figure 1. The unsmoothed curves include train/success_rate, eval/success_rate, train/pass@8, and eval/pass@4 for both runs. Training success increases toward 1, while training pass@8 reaches its ceiling earlier because it measures whether any of eight attempts succeeds. Evaluation pass@4 uses a different k and reaches 1 at iteration 20 for both runs. The evaluation success curves show p3's slower initial learning and similar final quality: 99.5% versus 99.0%. Train rewards describe accepted pre-update rollouts, potentially sampled by older policies in p3. Asynchronous evaluation measures the snapshot captured at its labeled iteration even if it finishes later; p3's final evaluation uses version 40 after ten skipped iterations.



Missing update diagnostics at skipped iterations remain gaps, not zeros.

Figure 2. Optimization and data health; reused from Part 3, Figure 2. Gradient norms stay finite, ranging from 0.0335–0.0946 in p2b and 0.0272–0.1355 in p3. IW means remain close to 1, while p3's larger maxima and standard deviations reveal a wider upper tail; Figure 3 supplies the lower-tail comparison. These are raw action-token ratios, so values outside PPO's clipping interval do not imply that clipping failed. Degenerate-group fractions rise as groups become uniformly successful, producing 16 skipped p2b iterations and ten skipped p3 iterations. Missing gradient, IW, and entropy diagnostics at those iterations remain gaps. The sampled action-token surprise proxy falls from 0.173 to 0.069 in p2b and from 0.169 to 0.054 in p3. It reflects retained nondegenerate groups and predictable formatting as well as decisions, so its decline alone does not demonstrate collapse.



Timing includes all 50 iterations; cumulative time excludes final evaluation drain.

Figure 3. Lower importance ratios and timing; reused from Part 3, Figure 3. IW minima reach 0.504 in p2b and 0.398 in p3, completing the wider-tail comparison without a runaway ratio tail in these runs. Raw step times preserve long iterations and short skipped ones; similar medians coexist with the lower asynchronous mean. Cumulative time sums all 50 iteration intervals and ends at 31.69 versus 26.78 minutes, before the final evaluation drain. The plotted learner-wait fraction is per iteration and can equal 1 on a nearly instantaneous skipped iteration. The table therefore summarizes waiting with time-weighted fractions, 66.3% versus 35.7%, rather than averaging those plotted fractions. Reduced waiting is consistent with overlapped sampling and background evaluation, while the differing GPU roles, updates, and retained batches remain relevant to interpretation.

Part 4.3: Importance Weighting under Higher Staleness

Setup and results

I compare the completed formal p43_number_search_async_stale4_iw and p43_number_search_async_stale4_noiw runs, using the provided configs unchanged. Both independently initialize Qwen/Qwen3.5-4B with LoRA rank 32, learning rate 10^{-5}, and seed 11768. Both use asynchronous GRPO, eight workers, eight groups of eight rollouts per iteration, queue capacity 16, maximum staleness 4, and 50 learner iterations. The configs differ only in loss and run name: IW uses PPO's clipped importance-weighted surrogate; noIW uses advantage-weighted cross-entropy. Each configuration was run once. Evaluation uses 50 validation tasks with four attempts each, at baseline and every ten iterations.

Metric	IW: PPO	noIW: weighted cross-entropy
Initial eval/success_rate	29.0%	37.5%
Iteration-20 eval/success_rate	54.0%	81.0%
Iteration-30 eval/success_rate	79.0%	99.0%
Final eval/success_rate	98.5%	98.5%
Initial eval/pass@4	72.0%	82.0%
Final eval/pass@4	100%	100%
Optimizer updates / skipped iterations	49 / 1	38 / 12
Accepted groups / associated rollouts	400 / 3,200	400 / 3,200
Retained training sequences	2,320	1,568
Retained action targets	394,900	253,012
Degenerate groups / accepted groups	110 / 400 (27.5%)	204 / 400 (51.0%)
Groups dropped for excessive staleness	0	0

Both runs learn successfully, finishing at 197 successful episodes out of 200, with at least one successful attempt on every validation task. noIW improves earlier in this realization: its evaluation success is 46%, 81%, 99%, 99%, and 98.5% at iterations 10 through 50, compared with 42.5%, 54%, 79%, 95.5%, and 98.5% for IW. Evaluation pass@4 reaches 1 at iteration 30 for noIW and iteration 40 for IW. The final decrease from 99% to 98.5% in noIW is one episode, not evidence of a substantial late failure.

More degeneracy in noIW accompanies its earlier success. All 12 skipped noIW iterations and the single skipped IW iteration contain only successful rollouts: their eight groups have identical rewards, hence zero centered advantages. They are skipped because they carry no GRPO learning signal. Mixed binary-reward groups retain all eight rollouts with nonzero centered advantages; there are 196 such groups in noIW and 290 in IW, explaining the different retained sequence totals. These counts cover groups accepted by the learner, excluding unused queued or in-flight work. Fewer updates here do not indicate an incomplete 50-iteration experiment.

Importance ratios, data health, and interpretation

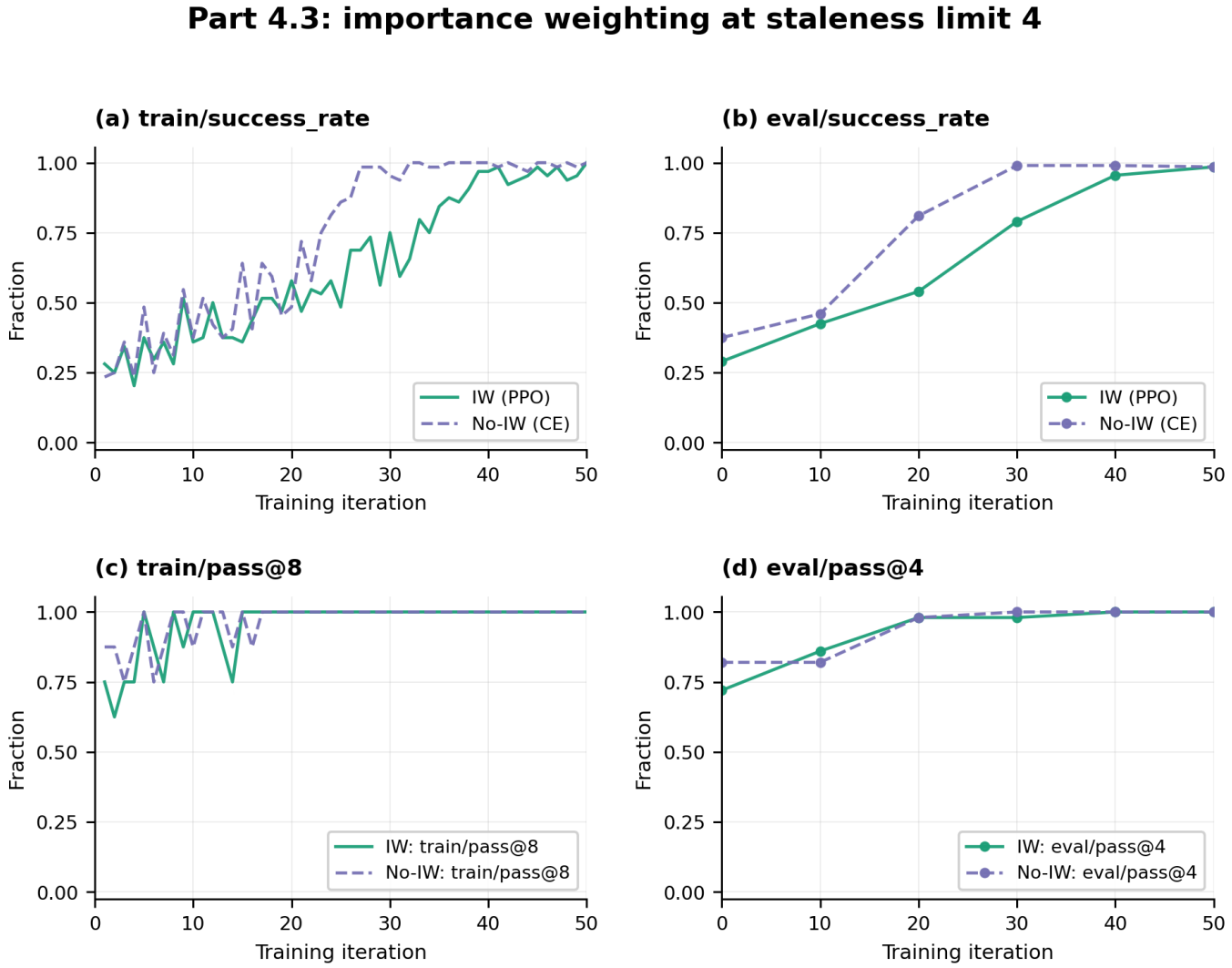
The logged action-token ratio is rho = exp(log_pi_train - log_pi_sample). Across updates, IW mean ratios range from 0.99935 to 1.00064 and noIW means from 0.99801 to 1.00075. A mean near 1 therefore describes both runs, but conceals tail differences: IW reaches a minimum of 0.1402, maximum of 2.449, and peak within-update standard deviation of 0.05097; noIW reaches 0.07739, 3.780, and 0.08182, respectively. The latter run has a wider observed ratio range and larger peak dispersion, despite its earlier reward improvement and identical final success. These ratios measure sampling/training log-probability mismatch on retained action tokens; they do not directly measure a group's policy-version lag.

The noIW label describes its objective, not the absence of ratio diagnostics. to_tinker_datum supplies signed centered advantages as cross-entropy weights and does not supply sampling log-probabilities to that loss. The backend still returns training-time log-probabilities, allowing importance_weight_stats to compare them with the stored sampling values. Thus the noIW ratio curves are diagnostic only. Its advantage-weighted objective is a signed policy-gradient surrogate, so a negative reported cross-entropy loss is valid. In IW, PPO uses the ratios and clipping in its surrogate; the plotted raw ratios can lie outside the clipping interval without indicating that clipping failed.

Neither run drops an excessively stale group, but this does not establish that all accepted groups were sampled by the current policy. The filter rejects a group only when the current learner version minus its generation version exceeds 4; accepted groups can be older by up to four updates. Final sampler versions are 49 and 38, matching actual optimizer updates rather than the shared 50-iteration counter. Evaluations refer to the snapshot captured at their labeled iteration, even if asynchronous evaluation completes later. These logs provide the rejection counts and mismatch diagnostics, not a complete histogram of accepted version lags.

All recorded metrics are finite, and all logged abandoned-episode counts are zero. Gradient norms range from 0.02935 to 0.12584 for IW and 0.03797 to 0.14788 for noIW. Action-token surprise and mean absolute advantage generally decrease as success rises, but describe retained nondegenerate groups and weight episodes by their action-token counts. They are not unconditional episode-level entropy or advantage statistics. Diagnostics requiring a training batch are absent on skipped iterations; zero retained sequences is a real observation rather than a missing value.

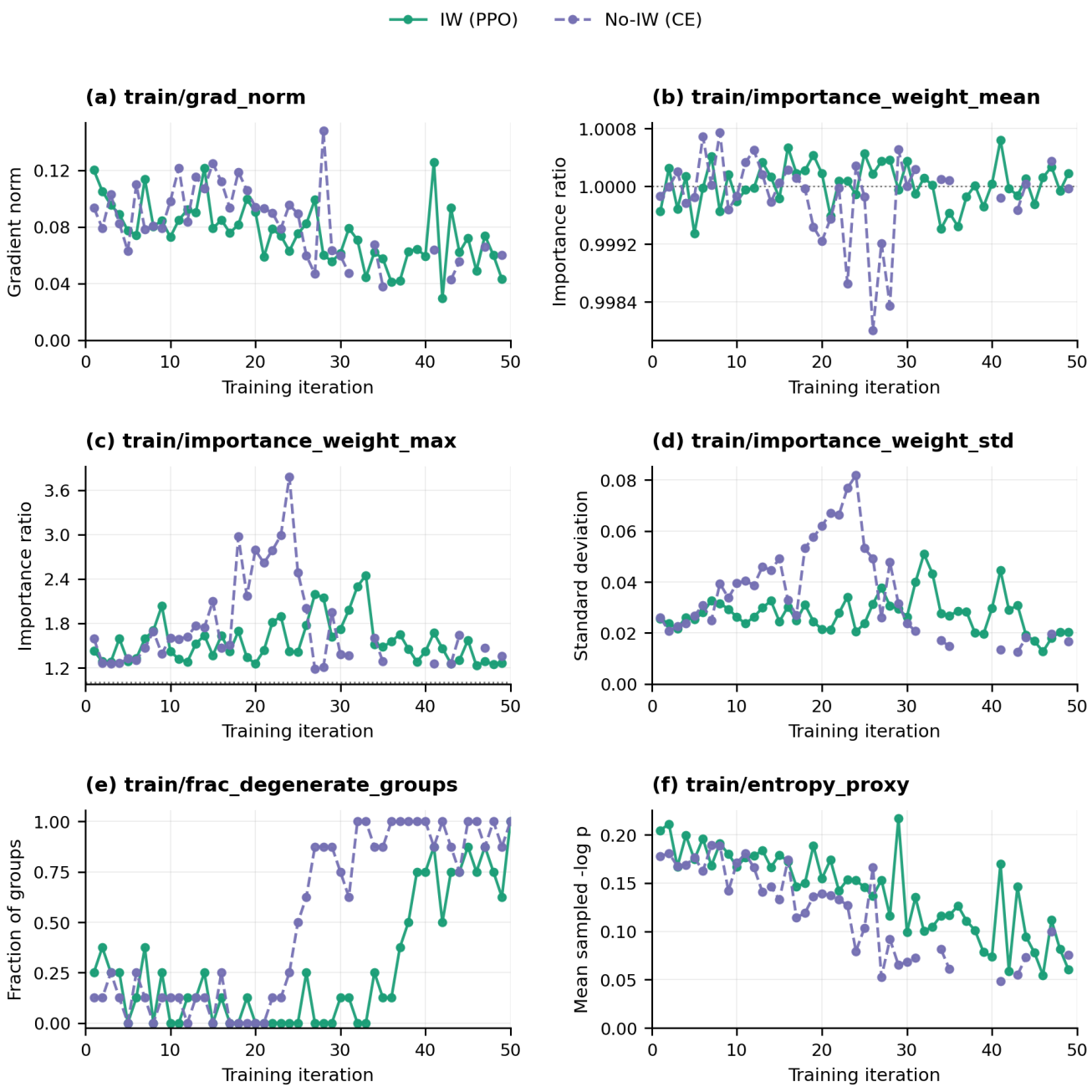
This ablation does not show that importance weighting is always required or always dispensable. The specified change replaces the entire PPO surrogate with weighted log-likelihood, removing its ratio dependence and clipping together. Different baseline success rates, stochastic trajectories, retained sample counts, and update counts also limit causal attribution. In these two runs, removing that correction did not prevent learning or reduce final measured success, although its ratio tails were wider. Multiple independent trials and richer lag measurements would be needed to generalize beyond this configuration; one run per configuration does not estimate variation across training runs.



Formal runs; train/pass@8 and eval/pass@4 are distinct quantities.

Figure 1. Reward quality versus learner iteration. Both unsmoothed training-success curves rise toward 1; noIW's evaluation-success curve rises earlier, reaching 99% at iteration 30 while IW is at 79%, before both finish at 98.5%. Training pass@8 is higher than per-episode success and approaches its ceiling earlier in both runs because it asks whether any of eight attempts succeeds. Evaluation pass@4 reaches 1 at iteration 30 for noIW and 40 for IW. Training metrics include all accepted groups, including degenerate ones, and describe pre-update rollouts that can come from older policies. Evaluation metrics describe the captured snapshot; the final snapshots are versions 49 and 38 rather than version 50.

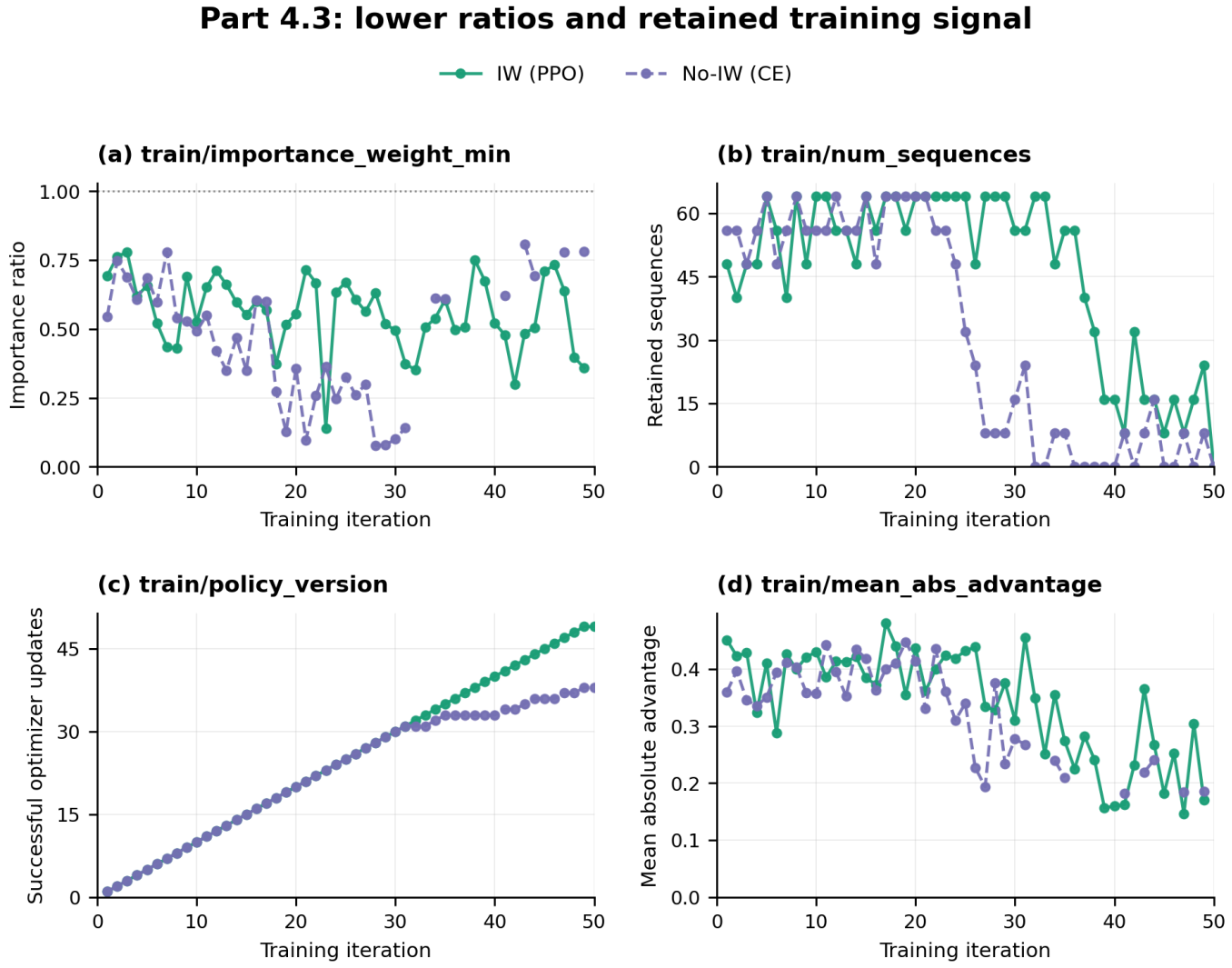
Part 4.3 · Diagnostic plots (2/3)



No-IW ratios are diagnostic only; skipped-update diagnostics stay missing.

Figure 2. Optimization, importance-ratio moments, and data health. Gradient norms stay finite in both runs, without a late explosion. The IW-mean panel stays close to 1 for both losses, while the IW-maximum and IW-standard-deviation panels reveal larger peaks in noIW: 3.780 and 0.08182 versus 2.449 and 0.05097. Ratios in noIW are diagnostics, and raw ratios in PPO are not bounded by its clipping interval. Degenerate-group fractions rise as training success approaches 1, more frequently producing all-degenerate batches in noIW. The sampled action-token surprise proxy generally declines in both runs, from 0.205 to 0.060 in IW and 0.178 to 0.076 in noIW, using the first and last nonempty batches. It includes predictable formatting and only retained groups, so its decline alone does not prove policy collapse. Missing gradient, IW, and entropy values remain gaps at skipped iterations.

Part 4.3 · Diagnostic plots (3/3)



Versions count updates, not iterations; advantage moments use retained action tokens.

Figure 3. Lower ratio tail, retained data, policy versions, and advantages. IW minima fall to 0.1402 with PPO and 0.07739 with weighted cross-entropy, complementing the upper-tail comparison. Retained sequence counts shrink as groups become uniformly successful; they sum to 2,320 for IW and 1,568 for noIW, with real zeros at one and 12 skipped iterations. Policy-version curves advance only after an optimizer update and finish at 49 and 38; noIW's late plateaus correspond to skipped all-success batches. Mean absolute advantage is the action-token-weighted magnitude among retained mixed groups: it generally declines from 0.451 to 0.171 for IW and 0.360 to 0.185 for noIW, using nonempty endpoint batches. It is absent at empty batches and must not be interpreted as the mean across all sampled episodes or as a direct measure of policy lag.

Part 5: TextCraft-Synth

Setup and training outcome

I ran the provided p5_textcraft_grpo configuration unchanged, once, from Qwen/Qwen3.5-4B with LoRA rank 32. Asynchronous GRPO used PPO, learning rate 10^{-5}, 32 rollout workers, queue capacity 16, 16 groups of eight rollouts per learner iteration, and maximum staleness 4. Thinking and preserved thinking history were enabled, with 512 generated tokens per turn and temperature 1.0. The run completed all 50 iterations and 50 optimizer updates without resuming; it consumed 800 groups and dropped zero groups for excessive staleness. The final sampler was tinker://model_71bf8a90/v50.

Periodic evaluation used the first 50 medium validation tasks, four attempts each. Its success rate rose from 11.5% at baseline to 17.5%, 29.0%, 44.0%, 54.0%, and 60.0% at iterations 10 through 50. Periodic pass@4 rose from 28% to 70%, with a slightly higher 72% at iteration 40; a one-task difference among 50 does not establish late deterioration. All iterations retained mixed-reward groups, yielding 3,578 training sequences and 36,315,974 retained action targets. The training log records 16 abandoned training episodes and seven abandoned periodic-evaluation episodes, all retained as failures.

Final full-split comparison

I evaluated the final adapter and the unmodified base model through the supplied standalone vLLM deployment. Each model received the full 213-task medium split and 136-task hard split, with four attempts per task: 852 and 544 episodes, respectively. Both used the same supplied agent, environment, chat template, thinking settings, generation limits, and scoring. The following table uses these complete splits, not the 50-task periodic subset.

Model	Split	success_rate	pass@4
Fine-tuned v50	Medium	62.21% (530/852)	77.46% (165/213)
Fine-tuned v50	Hard	0.18% (1/544)	0.74% (1/136)
Base Qwen3.5-4B	Medium	11.15% (95/852)	27.70% (59/213)
Base Qwen3.5-4B	Hard	0.00% (0/544)	0.00% (0/136)

On medium, fine-tuning improves per-episode success by 51.06 percentage points and pass@4 by 49.77 points. The mean completed environment-action count also falls from 61.83 to 49.48, consistent with more effective behavior, although this average mixes successes, budget-exhausted failures, and early-abandoned episodes and is not a success-only efficiency measure. On hard, both models remain near zero: the experiment shows strong improvement on the training difficulty but does not demonstrate strong transfer under the supplied harder-split budget. One training run and one evaluation batch per condition do not estimate variation across independent training runs, and the infrastructure failures below limit an infrastructure-free interpretation.

Failure accounting and the hard split

Every abandoned episode remains in the original denominator with reward zero; none was removed or replaced. Fine-tuned medium has 18 abandoned episodes: 17 HTTP 408 responses and one parser TypeError. Fine-tuned hard has 37: 15 HTTP 502 responses, 15 RemoteProtocolError, four ConnectError, two ReadError, and one parser TypeError. Base medium has none; base hard has one HTTP 502 and one parser TypeError. Thus communication failures and malformed model output are reported separately. All four commands reached their final result JSON. The two base evaluations used a progress-only wrapper around the supplied evaluator: the wrapper changed no tasks, model settings, concurrency, scoring, or retry rules and added no sampling calls.

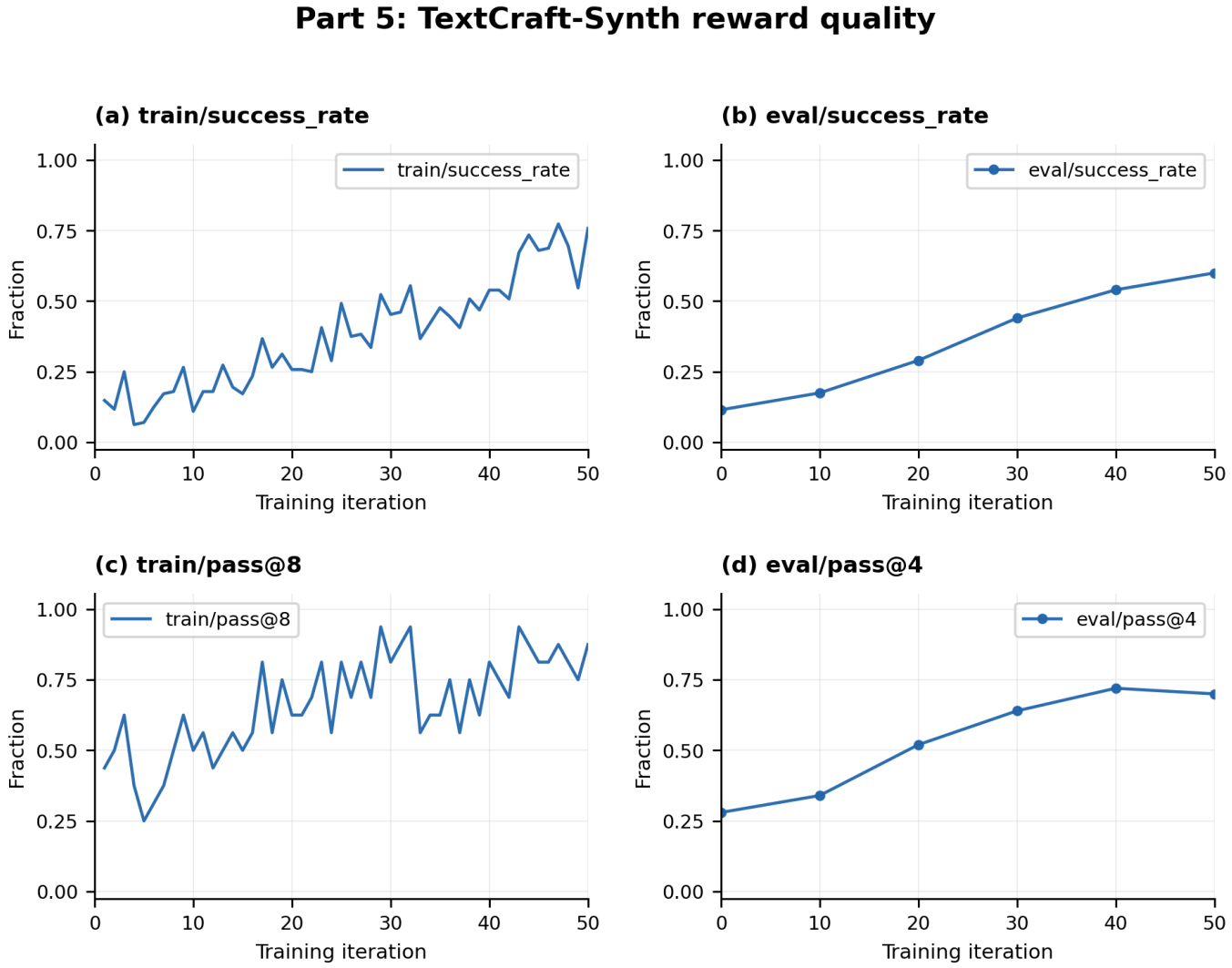
The instructor's Piazza clarification, “Part 5: one HTTP 408 during hard-split evaluation after upstream fix,” permits reporting an aborted episode as a failure and identifies a known hard-split issue. I retain and disclose the counts above, including the larger number of communication failures in this run, rather than treating that guidance as evidence that the failures had no effect. The reported success rates are the observed failure-inclusive outcomes, not estimates with infrastructure failures silently corrected. The earlier evaluation-endpoint patch was already present; evaluation used the direct vLLM endpoint rather than a stopped Tinker training session.

A separate static inspection of the supplied data reveals a concrete budget constraint. Every craftable item has one recipe, initial inventories contain only base materials, and one craft call produces only one item type, even when its quantity is batched. Counting the distinct required craftable ancestors therefore gives a lower bound on craft calls; the mandatory finish() adds one more action. For 84 of 136 hard tasks, this lower bound exceeds the fixed 75-action budget; no medium validation task violates that bound. This permits shared intermediates and batching, and even assumes perfect recipe knowledge and no wasted queries. It does not prove that the other 52 hard tasks are impossible or explain every failure, and I cannot identify it with certainty as the instructor's undisclosed root cause. The original datasets, environment, and budgets were left unchanged for every reported run.

System diagnostics and exported adapter

The mean learner-iteration time was 772.13 s; all 50 intervals sum to 10.72 hours. From the baseline result to the final periodic evaluation, elapsed time was 10.98 hours, including a 15.21-minute final evaluation drain but excluding startup and baseline evaluation. Time-weighted learner waiting was 73.6%, consistent with substantial rollout and other non-update time; this residual is not GPU utilization. Zero stale rejections does not imply that every accepted group was on-policy: groups up to four updates old remain admissible.

The exported public repository is Marstbowl/scottyrl-a3-p5_textcraft_grpo, recorded as hf_model_id in the run's checkpoint manifest. Its adapter configuration identifies the correct Qwen base and rank 32; the uploaded adapter_model.safetensors SHA256 matches the same file extracted from the final v50 archive. The export used the supplied script's local-directory option, so it uploaded the already evaluated weights without another training-server download. The repository will remain public until grades are released.



Periodic evaluation: first 50 medium tasks, k=4; not the full-split comparison.

Figure 1. Reward quality versus learner iteration. These are the unsmoothed train/success_rate, eval/success_rate, train/pass@8, and eval/pass@4 curves from the hero run. Training success increases from 14.84% to 75.78%, with batch-to-batch variation; training pass@8 rises from 43.75% to 87.50%. Pass@8 measures whether any of eight attempts succeeds and must not be equated with episode success or evaluation pass@4. The periodic evaluation curves use only the first 50 medium tasks; success rises to 60%, while pass@4 reaches 72% then finishes at 70%. Training rewards describe accepted pre-update rollouts, potentially from older policies. Evaluations use the snapshot captured at their labeled iteration, even when they finish later. The full-split comparison appears in the four-row table.

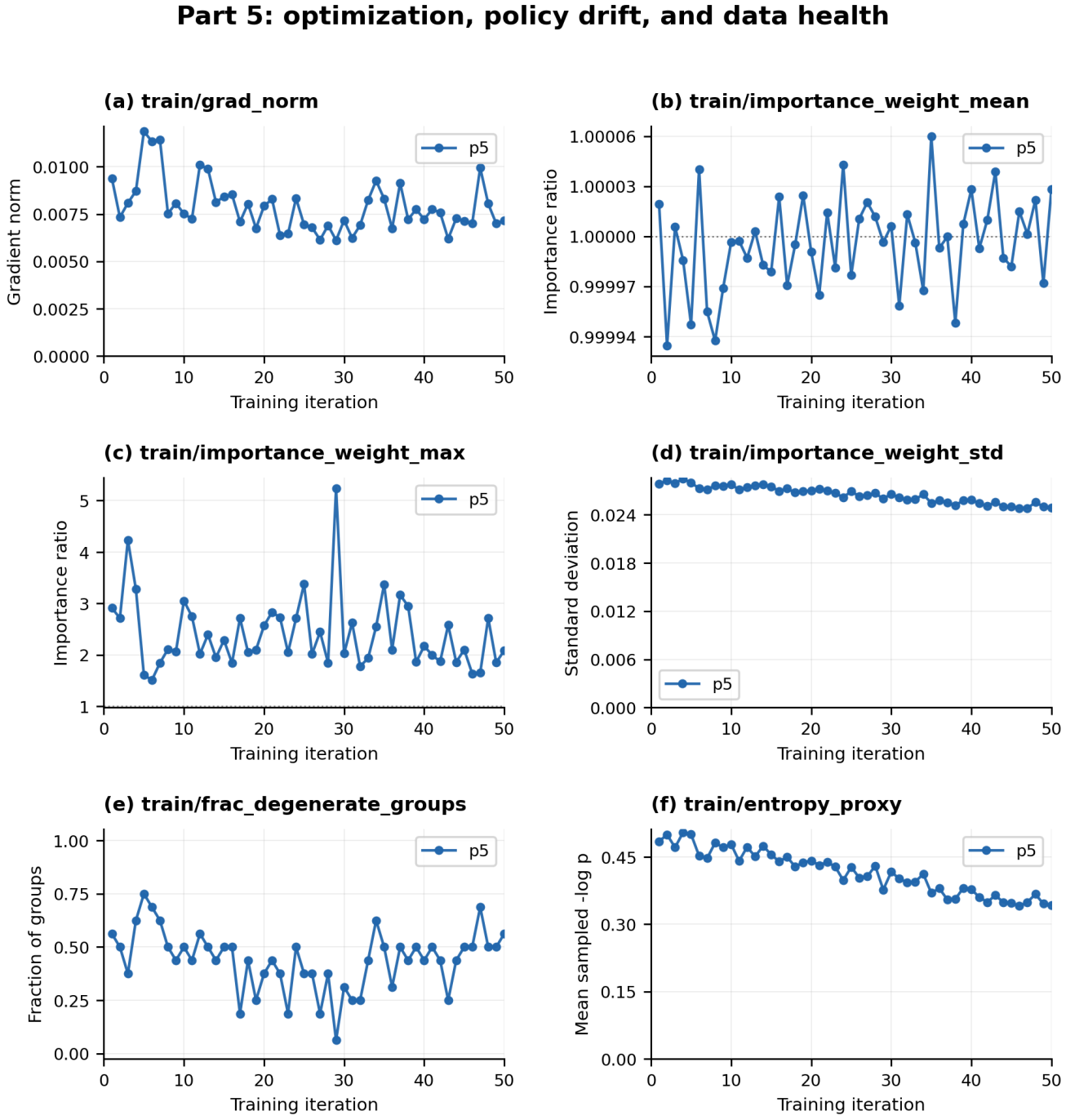
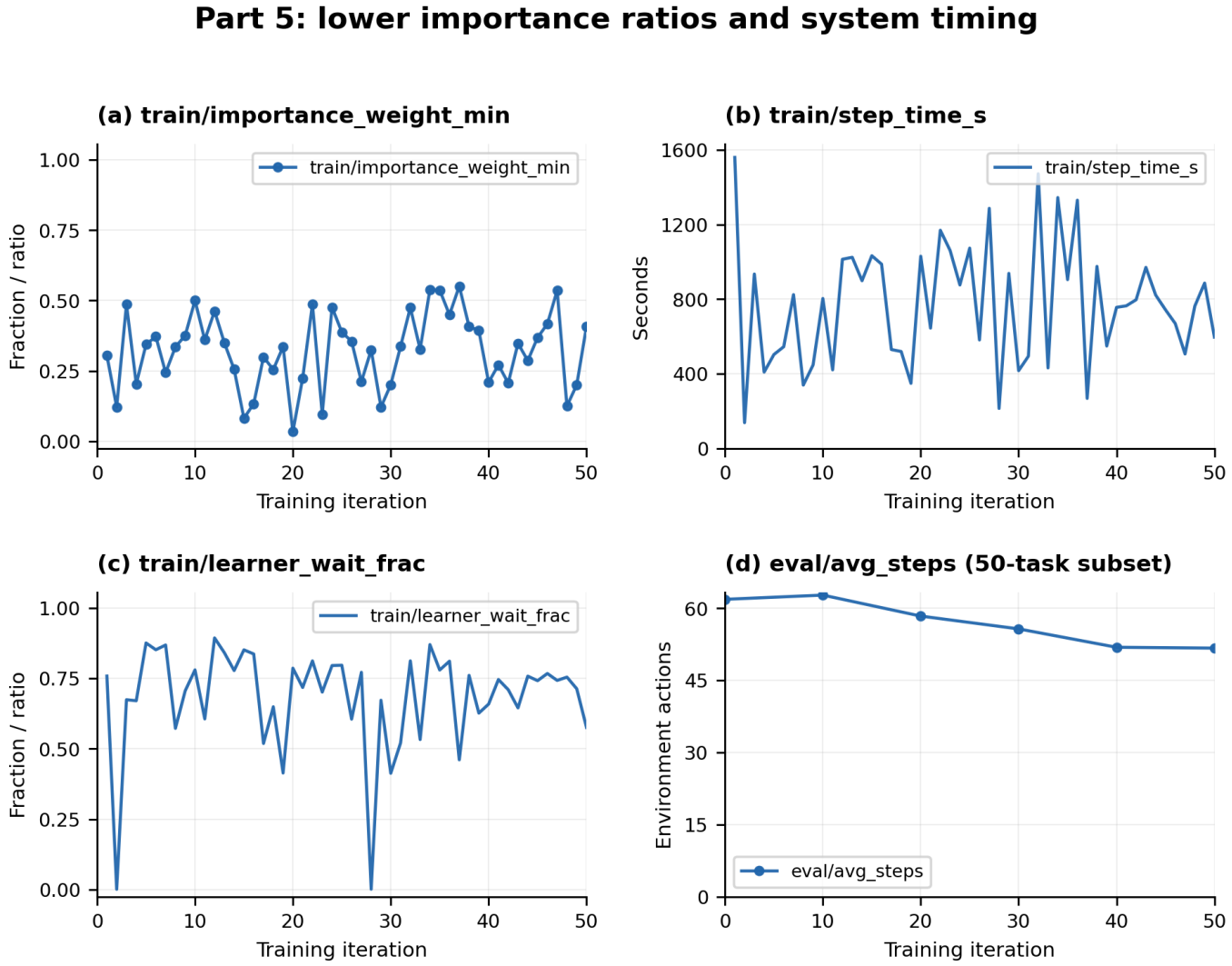


Figure 2. Optimization, importance ratios, and data health. Gradient norms remain finite at 0.00612–0.01187. IW means are very close to one (0.999935–1.000060), but the maximum reaches 5.240 and within-update standard deviation ranges from 0.02480 to 0.02848; the lower tail appears in Figure 3. A near-one mean therefore does not establish identical sampling and training distributions. These are raw action-token ratios, so PPO clipping does not bound the plotted maxima. Degenerate-group fractions vary between 6.25% and 75%, averaging 44.125%; none empties an entire batch. Such groups can be uniformly successful or uniformly unsuccessful, so degeneracy alone is not evidence of saturation. The sampled action-token surprise proxy decreases from 0.485 to 0.343. It measures retained nondegenerate groups and includes format tokens, so it is not full-distribution entropy or proof of collapse.



Unsmoothed data; learner intervals exclude the final evaluation drain.

Figure 3. Lower ratio tail, timing, and episode length. IW minima reach 0.03427, complementing the upper tail in Figure 2; mean ratios near one can conceal this mismatch. Step times range from 138.02 to 1,559.21 s, so the long run cannot be described by one constant step duration. The learner-wait fraction shows the raw per-iteration residual; the text instead summarizes waiting with its time-weighted 73.6%. Timing includes update/checkpoint and sampler-save overhead, and excludes the final evaluation drain. Periodic eval/avg_steps first rises from 61.85 to 62.73 and then falls to 51.70 as success improves. This includes failed and abandoned episodes, so it does not by itself establish a shorter successful crafting plan. All curves use original recorded values without smoothing, omitted failures, or added retries.